

CS 141 - Lab 3: Pointers in C

Exercise 1: Pointer Basics

Objective: Declare pointers, use `&` and `*`, and see how pointers relate to variables.

Task 1.1: Address and Value

1. Create `exercise1_1.c`
2. Declare an integer variable `x` and assign a value (e.g., 42).
3. Declare a pointer to `int` named `ptr` and make it point to `x` (e.g., `ptr = &x;`).
4. Using `printf`, display:
 - The value of `x`
 - The address of `x` (use `%p` and `&x`)
 - The value stored in `ptr` (the address)
 - The value pointed to by `ptr` (use `*ptr`)

Task 1.2: Modifying Through a Pointer

1. Create `exercise1_2.c`
2. Declare an integer `num` and a pointer `p` that points to `num`.
3. Assign a value to `num` and print it.
4. Change the value of `num` **only** by using `*p` (e.g., `*p = 100;`).
5. Print `num` again and show that it changed.

Expected Output (example):

```
Value of x: 42
Address of x: 0x7fff5fbff8ac
Value of ptr (address): 0x7fff5fbff8ac
Value at *ptr: 42
```

```
Before: num = 10
After using *p: num = 100
```

Hints

- Pointer declaration: `int *ptr;` (`ptr` can hold the address of an `int`).
- `&variable` gives the address of `variable`.
- `*ptr` gives the value stored at the address in `ptr`.
- Use `%p` to print pointer (address) values.

Exercise 2: Pointers and Arrays

Objective: Use the fact that an array name acts like a pointer to its first element.

Task 2.1: Array Name as Pointer

1. Create `exercise2_1.c`
2. Declare an integer array of size 5 and initialize it: `int arr[5] = {10, 20, 30, 40, 50};`
3. Declare a pointer: `int *p = arr;` (or `int *p = &arr[0];`).
4. Using a loop, print each element by:
 - Using array subscript: `arr[i]`
 - Using pointer and subscript: `p[i]`
 - Using pointer arithmetic: `*(p + i)`
5. Verify that all three give the same values.

Task 2.2: Traversing an Array with a Pointer

1. Create `exercise2_2.c`
2. Declare an array of 6 integers and read 6 values from the user.
3. Use a single pointer variable (e.g., `int *ptr = arr;`) and a loop to:
 - Find the sum of all elements (by advancing the pointer or using `ptr + i`).
 - Find the maximum element.
4. Print the sum and the maximum.

Expected Output (example):

```
arr[i]: 10 20 30 40 50
p[i]:   10 20 30 40 50
*(p+i): 10 20 30 40 50
```

```
Enter 6 numbers: 1 5 3 9 2 7
Sum: 27
Maximum: 9
```

Hints

- `arr` is the address of `arr[0]`. So `p = arr` makes `p` point to the first element.
- `*(p + i)` is the same as `p[i]` and `arr[i]`.
- You can advance the pointer: `ptr++` moves to the next element (increments by `sizeof(int)` bytes).

Exercise 3: Passing Pointers to Functions

Objective: Write functions that take pointers as parameters to read or modify variables.

Task 3.1: Function that Modifies a Variable via Pointer

1. Create `exercise3_1.c`
2. Write a function with this signature: `void doubleValue(int *ptr)`
 - Inside the function, double the value pointed to by `ptr` (e.g., `*ptr = *ptr * 2;`).
3. In `main`, declare an integer, print it, call `doubleValue(&variable)`, then print it again.
4. Show that the variable in `main` has changed.

Task 3.2: Function that Fills an Array

1. Create `exercise3_2.c`
2. Write a function: `void readArray(int *arr, int size)` that reads `size` integers from the user into `arr` (using a loop and `scanf("%d", &arr[i])` or `scanf("%d", arr + i)`).
3. In `main`, declare an array of 5 integers, call `readArray(arr, 5)`, then print the array.

Expected Output (example):

Before: `n = 5`

After `doubleValue`: `n = 10`

Enter 5 numbers: `1 2 3 4 5`

Array: `1 2 3 4 5`

Hints

- To modify a variable in another function, pass its address: `doubleValue(&n);`
- When you pass an array to a function, you pass its name (address of first element): `readArray(arr, 5);`
- Parameter `int *arr` in the function can be used like an array: `arr[i]` or `*(arr+i)`

Exercise 4: Simple Pointer Arithmetic

Objective: Use pointer arithmetic to move along an array and access elements.

Task 4.1: Print Array in Reverse Using a Pointer

1. Create `exercise4_1.c`
2. Declare an array of 7 integers and read 7 values from the user.
3. Set a pointer to point to the **last** element: `int *p = &arr[6];` (or `arr + 6`).
4. Using a loop and pointer arithmetic only (e.g., `p--`), print the array in reverse order.
5. Do not use array subscripts in the loop; use only `*p` and moving the pointer.

Task 4.2: Pointer Difference

1. Create `exercise4_2.c`
2. Declare an array: `int arr[] = {10, 20, 30, 40, 50};`
3. Set two pointers: `int *start = &arr[0];` and `int *end = &arr[4];`
4. Compute the number of elements between `start` and `end` using: `int count = end - start + 1;` (or similar).
5. Print the result and the first and last values using `*start` and `*end`.

Expected Output (example):

```
Enter 7 numbers: 1 2 3 4 5 6 7
Reverse: 7 6 5 4 3 2 1
```

```
First: 10, Last: 50
Number of elements: 5
```

Hints

- `p + 1` points to the next element; `p - 1` to the previous.
- `p++` and `p--` move the pointer by one element.
- For an array of size `n`, valid indices are `0` to `n-1`; the last element is at `&arr[n-1]` or `arr + n - 1`.

Challenge: Swap and Sum with Pointers

Objective: Use pointers to swap two integers and to sum an array via a function.

1. Create `challenge.c`
2. Implement a function `void swap(int *a, int *b)` that swaps the values of the two integers pointed to by `a` and `b`. In `main`, read two integers, print them, call `swap(&x, &y)`, then print them again.
3. Implement a function `int arraySum(int *arr, int size)` that returns the sum of the first `size` elements of the array pointed to by `arr`. In `main`, declare an array, read its elements, call `arraySum`, and print the result.

Expected Output (example):

```
Enter two numbers: 10 20
Before swap: 10 20
After swap: 20 10
```

```
Enter array size: 4
Enter 4 numbers: 1 2 3 4
Sum: 10
```